COMPSCI 687 - Final Project - Fall 2025

Aichen Qian, Linfeng Lyu, Wanqi Li

Option 2.

Select **two** RL algorithms that were not covered in class and evaluate each of them on at least **two** existing MDPs/domains. Since the point of this project is to implement these methods from scratch, you should not use existing code for the RL algorithms—though you may use existing code for the environments that you test.

Examples of algorithms that we have not covered in class that could be a good fit are the REINFORCE with Baseline algorithm (Section 13.4 of the RL book), the One-Step Actor-Critic algorithm for episodic tasks (Section 13.5 of the RL book), a model-based RL algorithm such as Prioritized Sweeping (Section 8.4 of the RL book), the Monte Carlo Tree Search (MCTS) algorithm (Section 8.11 of the RL book), the Episodic Semi-Gradient n-step SARSA algorithm (Section 10.2 of the RL book), the True Online SARSA(λ) algorithm (Section 12.7 of the RL book), and the PI^2 -CMA-ES algorithm from the paper *Path Integral Policy Improvement with Covariance Matrix Adaptation*.

If you decide to implement an algorithm that is not listed above, please check with the instructor and/or TAs to make sure it is appropriate for this course project.

In the write-up for this type of project, you should include:

- **(i)** A brief description of the methods and what they do;
- **(ii)** Pseudocode for the methods;
- **(iii)** A discussion of how you tuned their hyperparameters; and
- **(iv)** The corresponding experimental results. When analyzing the performance of each algorithm, you should present learning curves similar to those in the homework assignments.

Extra Credit

- **(i)** In case you chose to implement new RL algorithms, you might be planning to evaluate them in the toy domains introduced in class: the 687-GridWorld domain, the Cat-vs-Monsters domain, and the Mountain Car domain. One way to earn extra credit would be to implement other classic RL benchmark problems, such as Acrobot, Inverted Pendulum Swing-Up, or the Cartpole domain.
- **(ii)** You may choose to implement a *third* RL algorithm, as long as it is not a trivial modification or extension of one of the other algorithms you are implementing.

1 REINFORCE with Baseline algorithm

1.1 Brief Description

REINFORCE is a Monte Carlo policy gradient algorithm that optimizes parameterized policies to maximize expected total return. The agent interacts with the environment in episodes, collecting trajectories of states,

actions, and rewards. After each episode, the agent updates its policy parameters based on the observed returns using gradient ascent.

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha G_t \frac{\nabla \pi(A_t|S_t, \boldsymbol{\theta}_t)}{\pi(A_t|S_t, \boldsymbol{\theta}_t)}$$

where G_t is the return following time step t, α is the learning rate, and $\pi(a|s, \boldsymbol{\theta})$ is the policy parameterized by $\boldsymbol{\theta}$.

The REINFORCE with Baseline algorithm introduces a baseline function to the REINFORCE algorithm so that the equation becomes:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t + \alpha(G_t - b(S_t)) \frac{\nabla \pi(A_t|S_t, \boldsymbol{\theta}_t)}{\pi(A_t|S_t, \boldsymbol{\theta}_t)}$$

where $b(S_t)$ is the baseline function that depends only on the state S_t. The baseline function can be an function as long as it does not depend on the action and varies with states for MDPs. One natural choice is for baseline to be an estimate of the state-value function $\hat{v}(S_t, \boldsymbol{w})$ where $\boldsymbol{w}$ is the state-value weights that can be learned using Monte Carlo methods:

$$\boldsymbol{w}_{t+1} = \boldsymbol{w}_t + \alpha^{\boldsymbol{w}}(G_t - \hat{v}(S_t, \boldsymbol{w}_t)) \nabla \hat{v}(S_t, \boldsymbol{w}_t)$$

1.2 Pseudocode

<u>REINFORCE with Baseline</u>
Input: Differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Input: Differentiable state-value function $\hat{v}(s, \boldsymbol{w})$
Parameters: Step sizes $\alpha > 0$, $\alpha^{\boldsymbol{w}} > 0$
Initialize: Policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., 0)
Initialize: Value weights $\boldsymbol{w} \in \mathbb{R}^{d}$ (e.g., 0)
Loop: forever *(for each episode)*
  Generate an episode: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$
  For $t = 0, 1, \ldots, T-1$ **do**
    $G \leftarrow R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T$
    $\delta \leftarrow G - \hat{v}(S_t, \boldsymbol{w})$
    $\boldsymbol{w} \leftarrow \boldsymbol{w} + \alpha^{\boldsymbol{w}} \delta \nabla_{\boldsymbol{w}} \hat{v}(S_t, \boldsymbol{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t \delta \nabla_{\boldsymbol{\theta}} \ln \pi(A_t|S_t, \boldsymbol{\theta})$
  end for
end loop

1.3 Hyperparameter Tuning

For the REINFORCE with Baseline algorithm, we tuned the following hyperparameters:

- Step size α: We experimented with values in the range [0.00001, 0.1].
- Step size $\alpha^{\boldsymbol{w}}$: Similar to α, we tested values in the range [0.00001, 0.1].
- Discount factor γ: We evaluated values between 1 and 0.90 to experiment with different rate of learning.
- Policy Network Architecture: We tested different architectures including single-layer and multi-layer neural networks with varying numbers of neurons mainly from 4 to 128.
- Value Function Network Architecture: Similar to the policy network, we experimented with different architectures.

We performed the grid search process for searching the optimal hyperparameters on each of the four environments: CartPole, Blackjack, FrozenLake, and Taxi. For each environment, we evaluated the performance of the algorithm under different hyperparameter configurations by measuring the average return over 100 episodes.

1.4 Experimental Results

The goal for all experiments is to maximize and converge the final average return over 100 episodes.

1.4.1 Cart Pole

For Cart Pole environment, the set of hyperparameters that achieved getting reward of 500 after 2000 episode is when $\alpha = 0.01$, $\alpha^{\boldsymbol{w}} = 0.001$, $\gamma \geq 0.97$ with two layers in both policy and value networks that has at least 32 neurons. One of the best result we got is shown in Figure 1. This result also indicated the unstabling leanring process that happened frequently during our experiments. But with this set of hyperparameters, the agent is able to relearn quickly after a huge drop in performance.

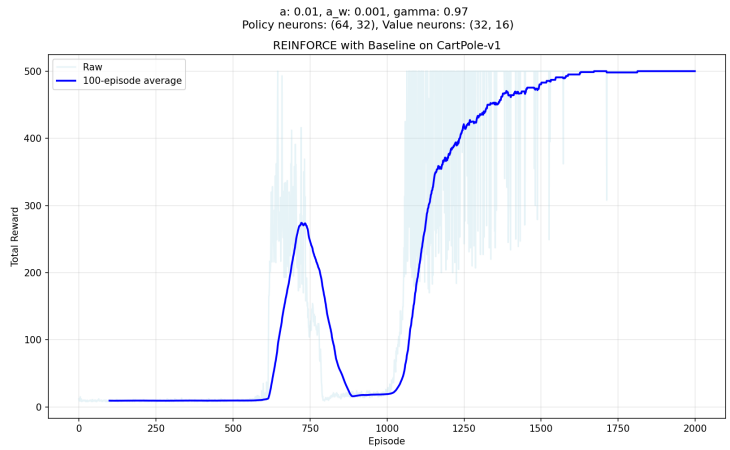

Figure 1: Learning curve for best result in CartPole environment.

One interesting result we found is that if α and $\alpha^{\boldsymbol{w}}$ are drasticly differnt from each other, the agent won't learn anything meaningful, example in Figure 2. However, if the policy network expends or value network to three layers, then there exist a sudden jump in the learning process, shown in Figure 3. In this case, the learning happens around 1250 episode. We found that increase the learning rate will bring the jump ealier.

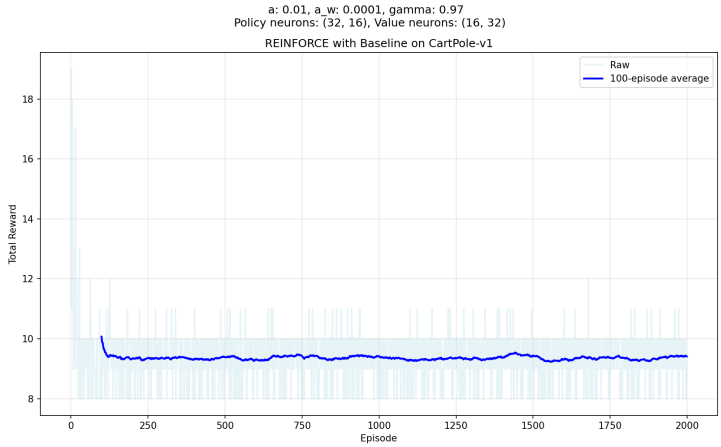

Figure 2: Learning curve for drastically different α and $\alpha^{\boldsymbol{w}}$ in CartPole.

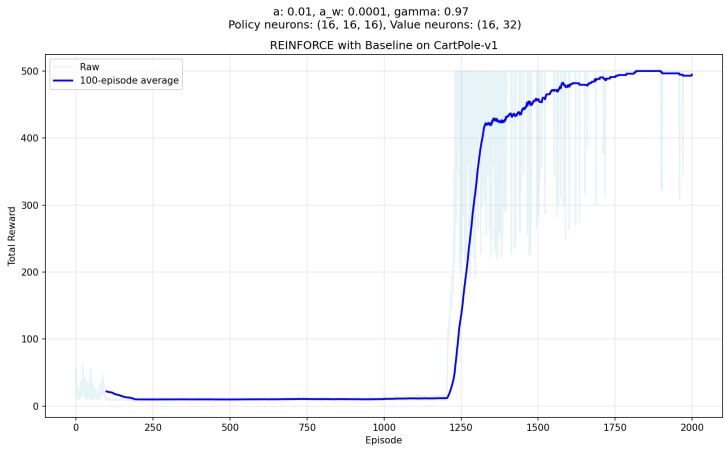

Figure 3: Learning curve for three-layer policy network in CartPole.

1.4.2 Blackjack

For Blackjack environment, we did not found any set of hyperparameters than can acchinve a stable reward. The average rewards oscillated between certain bounds. However we did found that with larger step sizes for α and $\alpha^{\boldsymbol{w}}$, the agent is able to get a slightly higher average reward. See Figure 4 and Figure 5 for comparison.

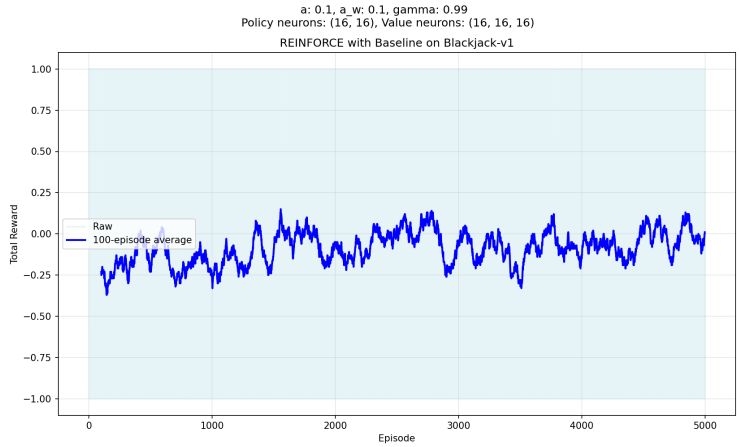

Figure 4: Learning curve for large step sizes in Blackjack.

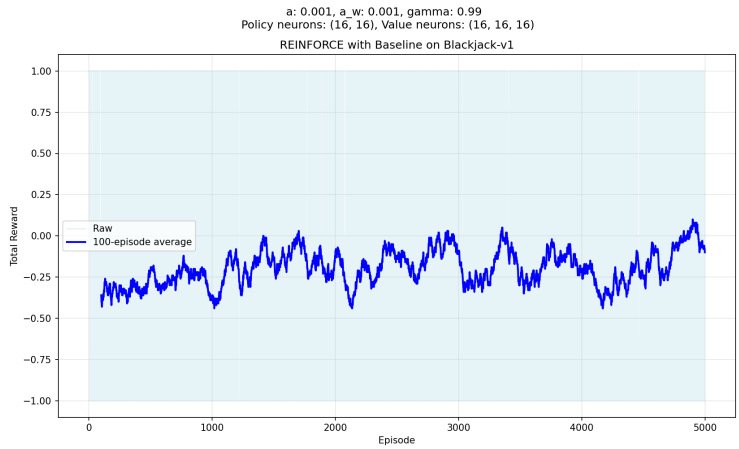

Figure 5: Learning curve for small step sizes in Blackjack.

The reason we think that this oscillation happens is because of the stochastic nature of the environment. As a poker game environment, the agent has to deal the the randomness of card dealing which makes the learning process difficcult and unstable.

1.4.3 Frozen Lake

For Frozen Lake environment, we found that with step sizes in the range of 0.1 to 0.01 for α and $\alpha^{\boldsymbol{w}}$, the agent is able to learn a stable policy that converge to reward of 1 consistently. An example learning curve is shown in Figure 6.

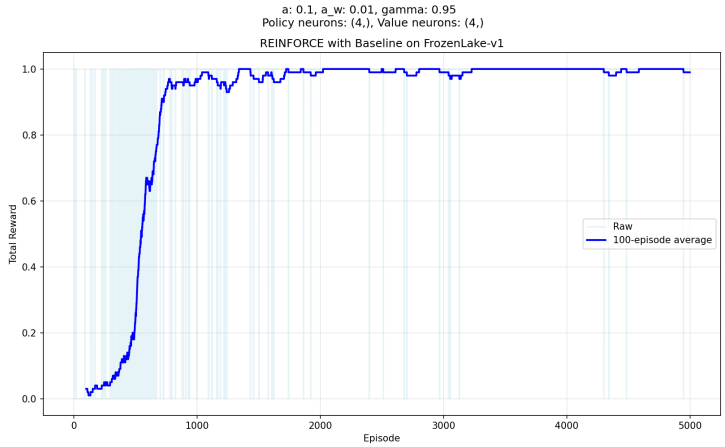

Figure 6: Learning curve for best result in FrozenLake environment.

We found that network architecture of one layer with 4 neurons is sufficient for both policy and value networks. As shown in Figure 7 increase neron size to 32 did not improve performance if not causing the learning to take place later.

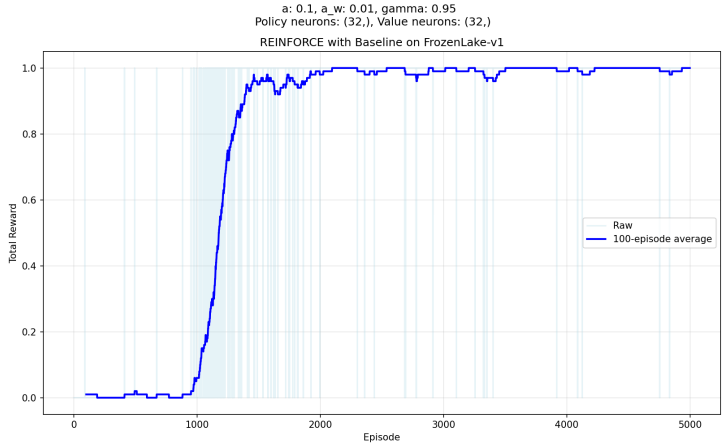

Figure 7: Learning curve for more neurons in FrozenLake environment.

We found that adding a layer to the value network and speed up the learning process. Figure 8 has the same neuron size as Figure 7 but with an additional layer in the value network. The learning happens around 400 episode instead of 1000.

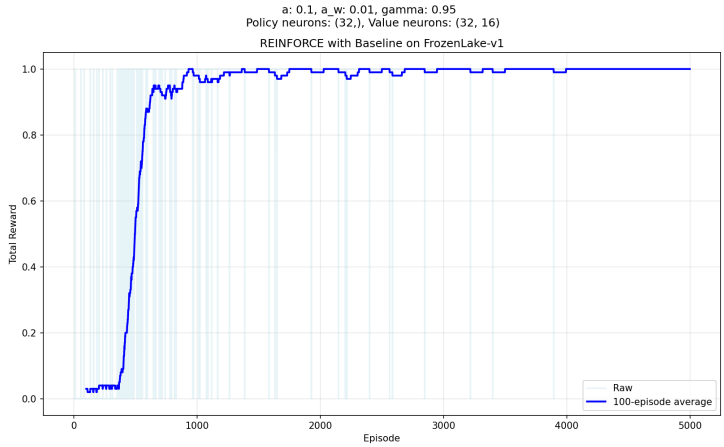

Figure 8: Learning curve for more layers and more neurons in FrozenLake environment.

1.4.4 Taxi

For the Taxi environment, we found that the maximum average reward the agent would achieve is -200. This means that the agent learned to randomly walk around the map without successfully picking up and dropping off the passenger. This is not the most desired behavior, but we think it is due to the complexity of the environment and the limited capacity of our policy and value networks. The agent found the a local optimum that maximizes its reward without actually completing the task. An example learning curve is shown in Figure 9.

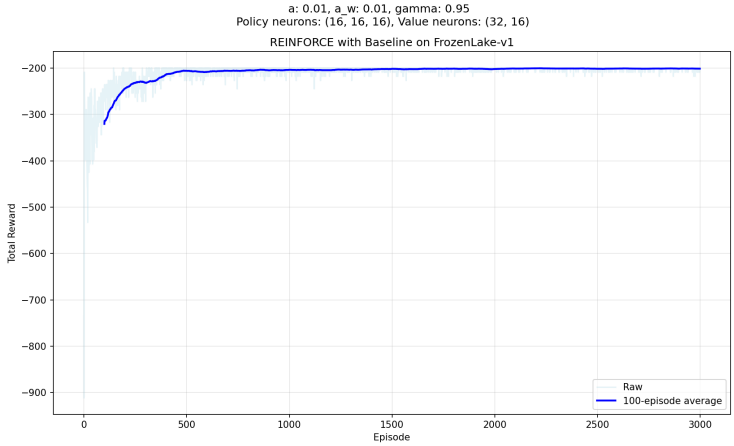

Figure 9: Learning curve for best result in Taxi environment. (Graph title misslabeled as FrozenLake, but it is for Taxi environment.)

We thought this could be improved by decrease policy step size and increase network capacity, but the results did not show significant improvement, shown in Figure 10.

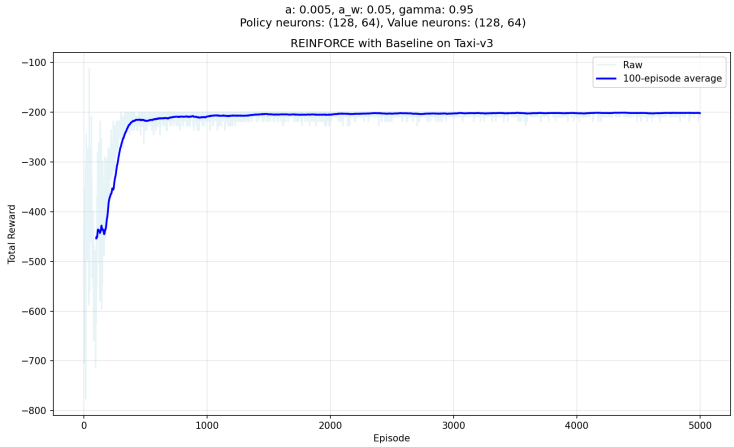

Figure 10: Learning curve for Taxi environment with larger networks and smaller step sizes.

We also tried other combinations of hyperparameters, but the results were very slow to converge so we determined that is not suitable for this environment. One example is shown in Figure 11.

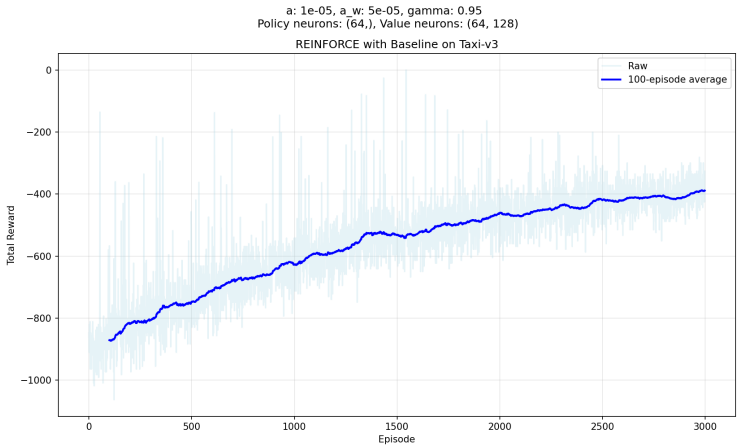

Figure 11: Learning curve for Taxi environment with different hyperparameters.

2 One-Step Actor-Critic algorithm (performed by Aichen Qian)

2.1 Brief Description

One-step Actor-Critic Algorithm. The One-step Actor-Critic algorithm is an on-policy reinforcement learning method that jointly learns a policy and a state-value function. The policy $\pi(a|s, \boldsymbol{\theta})$, called the actor, is a differentiable function parameterized by $\boldsymbol{\theta}$ and defines a probability distribution over actions given a state. The state-value function $\hat{v}(s, \boldsymbol{w})$, called the critic, is also differentiable and is parameterized by $\boldsymbol{w}$, estimating the expected discounted return from a state under the current policy. The algorithm uses separate step sizes $\alpha^{\boldsymbol{\theta}}$ and $\alpha^{\boldsymbol{w}}$ for updating the actor and the critic, respectively.

Episode Initialization. At the beginning of each episode, the environment is reset to obtain the initial state S. The auxiliary variable I is initialized to 1 and is used to apply discounting in the policy gradient

update. The parameters $\boldsymbol{\theta}$ and $\boldsymbol{w}$ are initialized before training, typically to zero or small random values.

Interaction and TD Error. At each time step, while the current state S is not terminal, an action A is sampled from the current policy $\pi(\cdot|S, \boldsymbol{\theta})$. After executing the action, the agent observes a reward R and the next state S'. The temporal-difference (TD) error is then computed as

$$\delta = R + \gamma \hat{v}(S', \boldsymbol{w}) - \hat{v}(S, \boldsymbol{w})$$

where γ is the discount factor. If the next state S' is terminal, the value estimate $\hat{v}(S', \boldsymbol{w})$ is set to zero. The TD error measures the difference between the observed return and the current value estimate.

Critic Update. The critic updates the value-function parameters using one-step TD learning:

$$\boldsymbol{w} \leftarrow \boldsymbol{w} + \alpha^{\boldsymbol{w}} \, \delta \, \nabla \hat{v}(S, \boldsymbol{w})$$

This update reduces the prediction error of the value function by moving $\hat{v}(S, \boldsymbol{w})$ toward the one-step bootstrapped target.

Actor Update. The actor updates the policy parameters using a policy gradient step:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}} \, I \, \delta \, \nabla \ln \pi(A|S, \boldsymbol{\theta})$$

We also have

$$\nabla \ln \pi(a|s, \boldsymbol{\theta}) = \frac{\nabla \pi(a|s, \boldsymbol{\theta})}{\pi(a|s, \boldsymbol{\theta})} = \mathbf{x}(s, a) - \sum_b \pi(b|s, \boldsymbol{\theta}) \mathbf{x}(s, b)$$

where $\mathbf{x}(s, a)$ is the feature vector for state-action pair (s, a).

Polynomial Feature. Polynomial features represent the state by using polynomial terms of the original state variables. Given a state vector $s = (s_1, s_2, \ldots, s_d)$, the feature vector may include constant terms, single variables, squared or higher-order terms, and cross-products such as $s_i s_j$. This allows a linear function approximator to model nonlinear relations between the state and the value or policy. However, the number of features grows quickly as the polynomial degree or the state dimension increases, which can increase computation cost and risk overfitting.

Fourier Feature. Fourier features map a continuous state into sinusoidal basis functions. For a normalized state $s \in [0, 1]^d$, a typical feature has the form $\cos(\pi k \cdot s)$ or $\sin(\pi k \cdot s)$, where k is an integer vector that controls the frequency. These features are global, meaning that a change in the state affects many feature values at the same time. Fourier features are well suited for approximating smooth value functions and are often used in reinforcement learning with continuous state spaces.

Tile Coding Feature. Tile coding is a sparse and binary feature representation. The state space is divided into multiple overlapping grids called tilings, each with a different offset. For a given state, exactly one tile in each tiling is active, and its corresponding feature value is set to one, while all other features are zero. As a result, similar states share some active tiles, leading to local generalization. Tile coding is simple, efficient, and commonly used with linear value function approximation.

RBF Feature. Radial Basis Function (RBF) features are defined using distance-based functions centered at selected points in the state space. Each feature typically has a Gaussian form,

$$\phi_i(s) = \exp\left(-\frac{\|s - c_i\|^2}{2\sigma_i^2}\right),$$

where c_i is the center and σ_i controls the width. The feature value is high when the state is close to the center and small when it is far away. RBF features provide smooth and local generalization, but their performance depends on the choice of centers and widths.

Coarse Coding Feature. Coarse coding represents the state using a set of overlapping regions in the state space. Each feature corresponds to one region and is active if the state lies inside that region. The regions are large, so each feature gives low-resolution information about the state. Multiple overlapping active features together provide generalization across nearby states. Coarse coding is conceptually similar to tile coding but is less structured and often described at a more general level.

Transition and Termination. After the updates, the discounting variable is updated as $I \leftarrow \gamma I$, and the state is updated as $S \leftarrow S'$. This process repeats until a terminal state is reached, at which point the episode ends. Through repeated episodes, the critic improves its value estimates, and the actor learns a policy that approximates the optimal policy.

2.2 Pseudocode

<u>One-step Actor–Critic</u>
Input: Differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Input: Differentiable state-value function $\hat{v}(s, \boldsymbol{w})$
Parameters: Step sizes $\alpha^{\boldsymbol{\theta}} > 0$, $\alpha^{\boldsymbol{w}} > 0$
Initialize: Policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., 0)
Initialize: Value weights $\boldsymbol{w} \in \mathbb{R}^{d}$ (e.g., 0)
Loop: forever *(for each episode)*
  Initialize S (initial state)
  $I \leftarrow 1$
  While S is not terminal **do**
    Sample $A \sim \pi(\cdot|S, \boldsymbol{\theta})$
    Take action A, observe S', R
    $\delta \leftarrow R + \gamma \hat{v}(S', \boldsymbol{w}) - \hat{v}(S, \boldsymbol{w})$ *(If S' is terminal, $\hat{v}(S', \boldsymbol{w}) = 0$)*
    $\boldsymbol{w} \leftarrow \boldsymbol{w} + \alpha^{\boldsymbol{w}}\, \delta \nabla_{\boldsymbol{w}} \hat{v}(S, \boldsymbol{w})$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha^{\boldsymbol{\theta}}\, I\, \delta \nabla_{\boldsymbol{\theta}} \ln \pi(A|S, \boldsymbol{\theta})$
    $I \leftarrow \gamma I$
    $S \leftarrow S'$
  end while
end loop

2.3 Hyperparameter Tuning

We systematically tuned hyperparameters for four environments (CartPole, Blackjack, FrozenLake, and Taxi). For each environment, we tested different combinations of learning rates for the actor (α_θ) and critic (α_w), feature representations (Polynomial, Fourier, Tile Coding, RBF, and Coarse Coding), and feature complexity (such as polynomial order, number of tilings, or number of centers). Each environment was evaluated under four configurations. We analyzed final performance and the stability of the temporal-difference (TD) error.

2.3.1 Cart Pole

Configurations Tested

Config	Feature	α_θ	α_w	Params	Final Reward
1	Polynomial	0.001	0.01	order=2	29.11
2	Polynomial	0.002	0.02	order=3	29.55
3	Fourier	0.001	0.01	order=2	**500.00**
4	RBF	0.001	0.01	centers=4, $\sigma = 0.3$	**500.00**

Key Findings Fourier and RBF features both achieved perfect performance, while polynomial features failed to converge. Fourier features converged faster, reaching the maximum reward after 1 episode, whereas RBF required about 700 episodes. Polynomial features oscillated heavily and never exceeded a reward of 100.

Here we find that for Fourier features, the reward curve quickly rises to the maximum of 500. Given the low dimensionality and high determinism of Cart Pole, as well as the strong approximation capability of the Fourier basis (order 2), this behavior is considered expected — the algorithm rapidly converges to a stable policy.

Analysis Low-order polynomial features were not expressive enough to model the nonlinear balance dynamics of CartPole. TD errors remained large and unstable, indicating a non-convergent critic. In contrast, Fourier features captured the oscillatory nature of the pole angle effectively, leading to stable learning and near-zero TD errors after convergence. RBF features also achieved stable performance but required more exploration due to localized generalization.

The main takeaway is that, for CartPole, feature representation dominated learning rate selection. Poor features could not be compensated by tuning α_θ or α_w.

2.3.2 Blackjack

Configurations Tested

Config	Feature	α_θ	α_w	Params	Final Reward
1	Polynomial	0.05	0.10	order=2	**0.07**
2	Polynomial	0.10	0.20	order=3	-0.17
3	Tile Coding	0.05	0.10	4×3	-0.26
4	Fourier	0.05	0.10	order=2	-0.12

Key Findings All configurations showed high variance due to stochastic card draws and sparse rewards. The best result was achieved by the simplest representation, a second-order polynomial with relatively high learning rates.

Analysis A second-order polynomial provided enough capacity to model interactions between player sum, dealer card, and usable ace without overfitting. Increasing polynomial order destabilized learning because the number of parameters grew too large for the small discrete state space. Tile coding performed poorly because it re-discretized already discrete variables, which led to information loss.

2.3.3 Frozen Lake

Configurations Tested

Config	Feature	α_θ	α_w	Params	Final Reward
1	Tile Coding	0.01	0.02	4×4	0.98
2	Tile Coding	0.02	0.05	6×4	**1.00**
3	Coarse Coding	0.01	0.02	circles=50	0.99
4	Polynomial	0.01	0.02	order=2	0.18

Key Findings Tile coding with higher resolution achieved perfect performance and converged the fastest. Polynomial features almost completely failed to learn the task.

Analysis FrozenLake is a grid world with stochastic transitions, making spatial generalization essential. Tile coding naturally matched this structure and produced stable, monotonic learning with rapidly decreasing TD errors. Increasing the number of tilings improved generalization across nearby states. Polynomial features treated coordinates as continuous variables and could not represent discrete transitions caused by slipping dynamics, leading to biased TD errors and near-random behavior.

2.3.4 Taxi

Configurations Tested

Config	Feature	α_θ	α_w	Params	Final Reward
1	Tile Coding	0.05	0.10	6×3	**-200.18**
2	Polynomial	0.05	0.10	order=2	**-200.00**
3	Coarse Coding	0.05	0.10	circles=50	-436.43
4	RBF	0.05	0.10	centers=3, $\sigma = 0.4$	**-200.00**

Key Findings Tile coding, polynomial, and RBF features all achieved near-optimal performance. Coarse coding failed completely.

Analysis Taxi combines spatial navigation with relational reasoning between taxi, passenger, and destination. Polynomial features successfully modeled pairwise relations, while RBF features captured distance-based structure smoothly. Tile coding worked well due to structured discretization of the state space. Randomly placed coarse coding features caused uneven coverage of states, leading to unstable TD errors and poor policies. Dense step penalties allowed the use of high learning rates without divergence.

2.3.5 General Insights

Feature Representation Across all environments, feature choice had a much larger impact than learning rate tuning. Mismatched representations led to large performance gaps that could not be recovered through hyperparameter adjustment.

Learning Rates Optimal learning rates depended on reward structure. Sparse-reward environments benefited from higher learning rates, while smooth control tasks required smaller values for stable updates.

TD Error Diagnostics TD error behavior was a reliable indicator of learning health. Decreasing variance and oscillation around zero signaled convergence, while persistent bias or large variance indicated feature mismatch or instability.

2.4 Experimental Results

2.4.1 Cart Pole

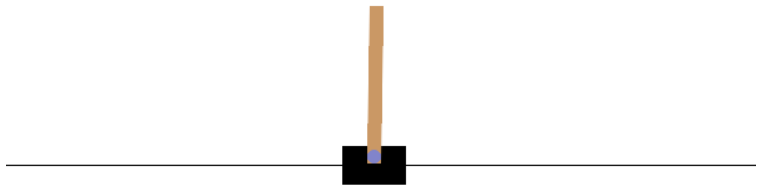

Figure 12: The visualization of Cart Pole environment.

Using Fourier and RBF feature representations, the one-step actor-critic algorithm consistently achieved the maximum reward of 500. Fourier features converged faster, reaching the optimal performance within 1 episode, whereas RBF features required around 700 episodes. In contrast, polynomial feature representations of order 2 and 3 failed to converge, with final rewards remaining below 30.

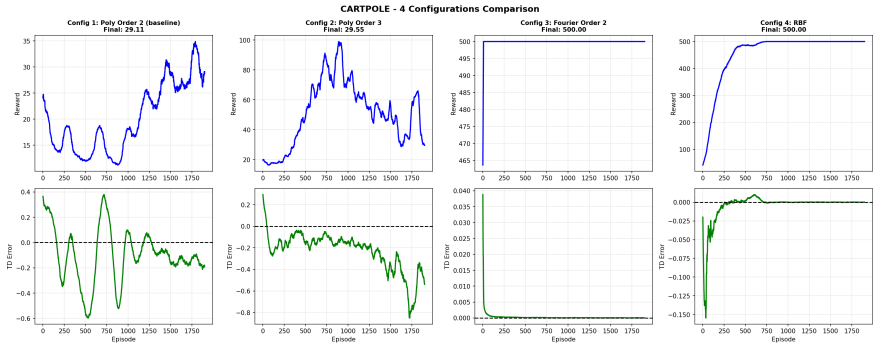

Figure 13: The plot of Cart Pole environment with different hyperparameter tuning.

The TD errors under Fourier and RBF features decreased rapidly during training and stabilized near zero (within ± 0.02) after convergence. Polynomial features exhibited large and persistent TD error oscillations, indicating unstable value estimation.

2.4.2 Blackjack

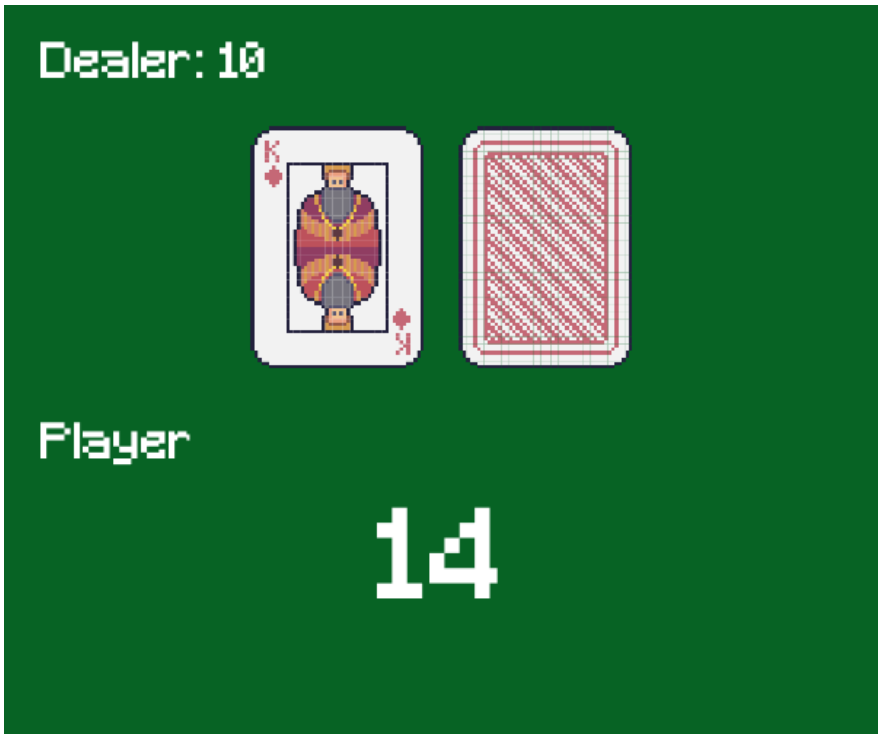

Figure 14: The visualization of Blackjack environment.

In the Blackjack environment, all configurations showed high variance due to stochastic card draws and sparse terminal rewards. The best-performing configuration used a second-order polynomial feature representation with relatively high learning rates, achieving a slightly positive average reward of 0.07.

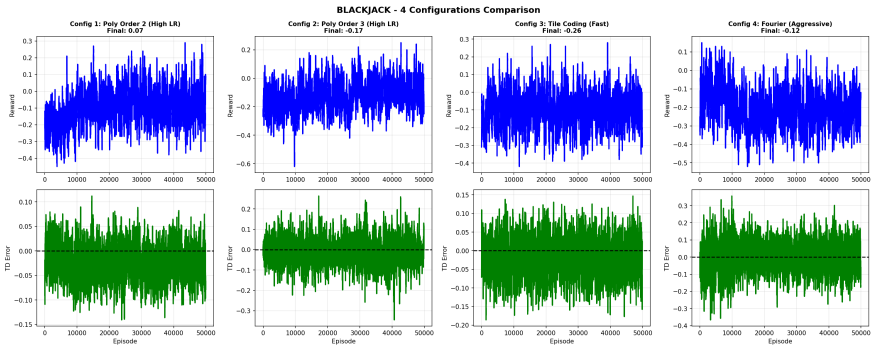

Figure 15: The plot of Blackjack environment with different hyperparameter tuning.

Higher-order polynomial features and tile coding resulted in negative average rewards. TD errors remained noisy across all configurations, typically within the range ± 0.1 to ± 0.2, even after extended training. Despite the variance, the TD errors in the best configuration were centered around zero, indicating an unbiased but noisy value estimate.

We think this one-step actor-critic algorithm is not very suitable for the Blackjack environment, as the

high variance in rewards and sparse feedback make it difficult for the critic to learn accurate value estimates. This leads to unstable policy updates by the actor, resulting in suboptimal performance.

2.4.3 Frozen Lake

Figure 16: The visualization of Frozen Lake environment.

Tile coding and coarse coding both achieved near-optimal performance in FrozenLake. The best configuration, tile coding with 6×4 tilings, reached a success rate of 1.00 and converged within approximately 1000 episodes. A coarser tile coding scheme (4×4) converged more slowly and attained a final reward of 0.98.

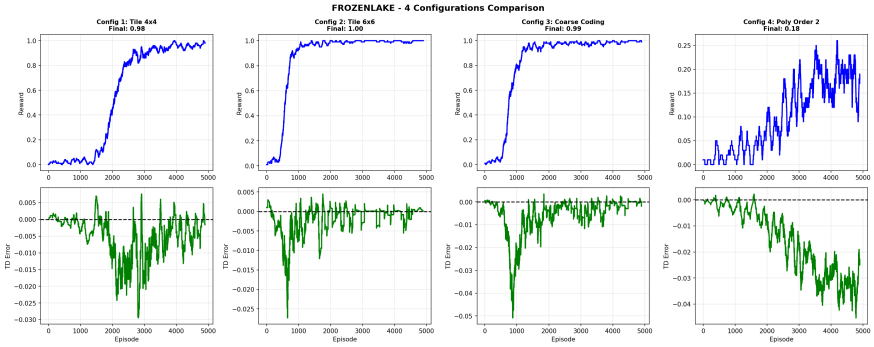

Figure 17: The plot of Frozen Lake environment with different hyperparameter tuning.

Polynomial features performed poorly, with final performance around 0.18 and no clear convergence trend. Correspondingly, TD errors for tile and coarse coding steadily decreased and converged to small values (around ± 0.005), while polynomial features showed biased and unstable TD errors throughout training.

2.4.4 Taxi

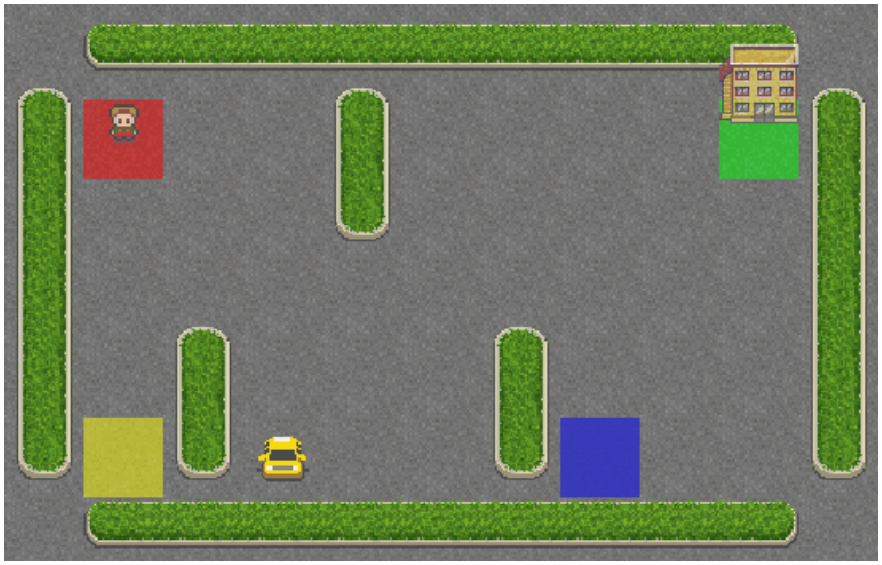

Figure 18: The visualization of Taxi environment.

In the Taxi environment, tile coding, polynomial, and RBF feature representations all achieved near-optimal performance, with final rewards around -200. Tile coding converged the fastest, typically reaching its plateau within 500 episodes. Polynomial and RBF features converged slightly later but displayed smooth and stable learning curves.

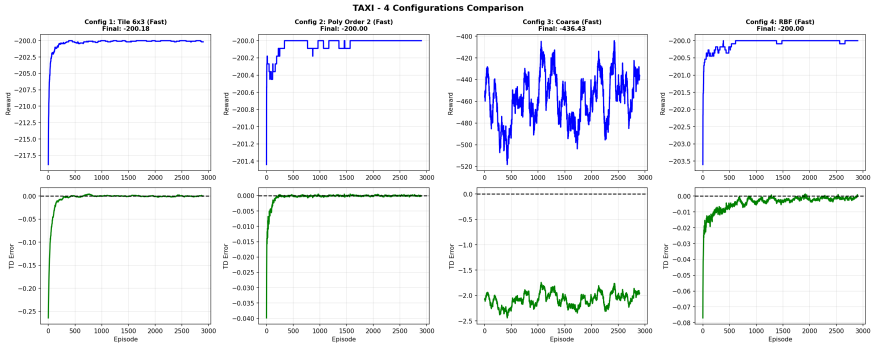

Figure 19: The plot of Taxi environment with different hyperparameter tuning.

Coarse coding failed to learn an effective policy, resulting in a much lower final reward of approximately -436. For the successful configurations, TD errors decreased quickly and remained close to zero (within ± 0.005). The failed configuration exhibited high-variance TD errors that did not stabilize over training.

The reward is strange as it is negative. In taxi environment, the agent receives a reward of -1 for each time step until it successfully delivers the passenger to the destination, at which point it receives a reward of $+20$. Therefore, we tried to use onehot encoding to represent the state, hoping to capture the discrete nature of the environment better. However, this approach did not yield significant improvements in performance compared to other feature representations as shown below.

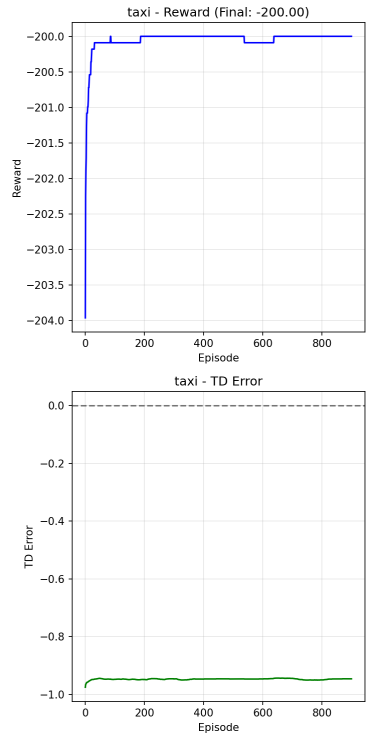

Figure 20: The plot of Taxi environment with onehot encoding feature representation.

We think maybe one-step actor-critic algorithm is not very suitable for the Taxi environment.

3 Monte Carlo Tree Search algorithm

3.1 Brief Description

We implemented the Monte Carlo Tree Search (MCTS) algorithm, a decision-time planning method that builds a search tree to estimate the value of actions from the current state. Our implementation follows the standard four phases:

- **Selection:** Starting from the root, we traverse the tree by selecting child nodes that maximize the Upper Confidence Bound (UCB1) statistic:

$$UCB(n) = \frac{Q(n)}{N(n)} + c\sqrt{\frac{\ln N_{parent}}{N(n)}}$$

  where $Q(n)$ is the total reward of node n, $N(n)$ is the visit count, and c is the exploration constant.

- **Expansion:** Once a leaf node is reached, if it is not terminal, we expand it by adding a child node corresponding to an untried action.

- **Simulation (Rollout):** From the newly expanded node, we perform a rollout using a random policy until a terminal state is reached or a maximum depth is exceeded.

- **Backpropagation:** The cumulative reward obtained from the rollout is propagated back up the tree, updating the visit counts and value estimates of all traversed nodes.

We tailored the simulation step for two distinct environment types:

- **Deterministic (CartPole-v1):** We utilized an action-sequence replay approach. To reach a specific node during simulation, the agent resets the environment to the root state and replays the sequence of actions stored in the tree branch.
- **Stochastic (Blackjack-v1):** We utilized a state-copying approach. The environment instance itself is deep-copied at the expansion phase to ensure that rollouts do not corrupt the true environment state.

3.2 Pseudocode

```
MONTE CARLO TREE SEARCH (MCTS)
Input: Environment E, Current State S_0, Action Space A
Parameters: Simulations per step N_sim, Exploration constant c
Output: Best Action a*

Create root node v_0 with state S_0
For i = 1 to N_sim do
    v_l ← TreePolicy(v_0) // Selection phase using UCB1
    If v_l is not terminal then
        v_c ← Expand(v_l) // Expansion phase
        Δ ← DefaultPolicy(v_c) // Simulation/Rollout phase
    End If
    Backpropagate(v_c, Δ) // Backpropagation phase
End For
Return arg max_{a∈A} N(child(v_0, a))
```

3.3 Hyperparameter Tuning

We evaluated the agent on two Gymnasium environments: `CartPole-v1` and `Blackjack-v1`. The primary hyperparameters tuned were the number of MCTS simulations per decision step (n_{sim}) and the UCB exploration constant (c).

Simulation Budget (n_{sim}) We varied n_{sim} to analyze the trade-off between computational cost and decision quality.

- For **CartPole**, we tested $n_{sim} \in \{10, 25, 50, 100, 200\}$.
- For **Blackjack**, we tested $n_{sim} \in \{25, 50, 100, 200, 500\}$.

As expected, higher simulation counts generally led to better performance but increased the runtime linearly. For CartPole, performance stabilized around 50 simulations, while Blackjack required more extensive search (100+) to reliably outperform the baseline due to the high variance in card distribution.

Exploration Constant (c) We tested $c \in \{0.5, 1.0, 1.41, 2.0\}$. The theoretical value of $\sqrt{2} \approx 1.41$ provided the most robust balance between exploring new actions and exploiting known high-value paths across both domains.

3.4 Experimental Results

It is important to note that unlike the Reinforcement Learning algorithms implemented in Option 2, our MCTS agent does not "learn" parameters across episodes. MCTS is a decision-time planning algorithm where each episode represents an independent experiment.

The "intelligence" of the agent is determined by the computational budget (number of simulations, N_{sim}) allocated to each decision step, rather than accumulated experience over time. Consequently, the performance graphs (e.g., Figure 21) do not exhibit the typical rising trend of a learning curve. Instead, they show a flat, stable performance characteristic of the fixed simulation budget. We increased the number of episodes solely to smooth out statistical fluctuations arising from the stochastic nature of the rollouts, ensuring the reported average reward is statistically significant.

3.4.1 Cart Pole

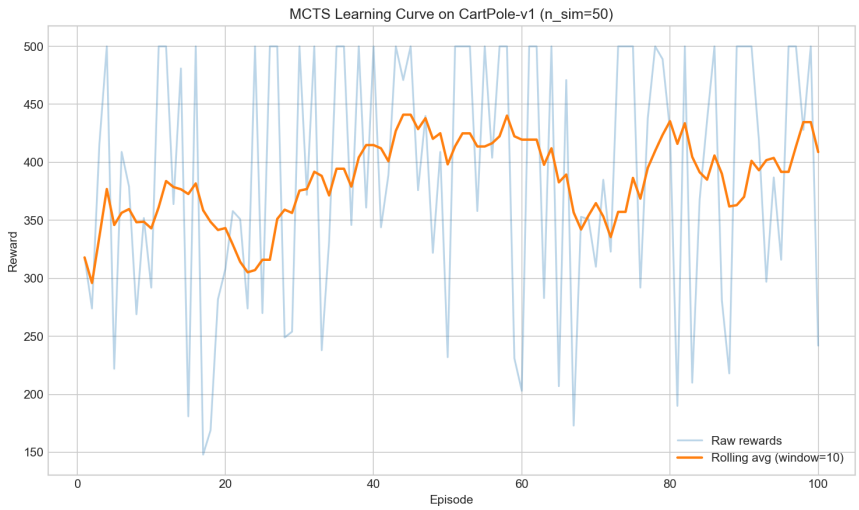

Figure 21: Learning curve for MCTS on CartPole-v1. The agent consistently solves the environment (reward 500) without explicit training, relying solely on planning.

The MCTS agent demonstrated superior performance on the deterministic CartPole-v1 environment. Unlike reinforcement learning algorithms that require episodes to learn weights, MCTS achieved near-optimal rewards almost immediately by planning ahead. Figure 21 shows the agent maintaining the maximum reward threshold.

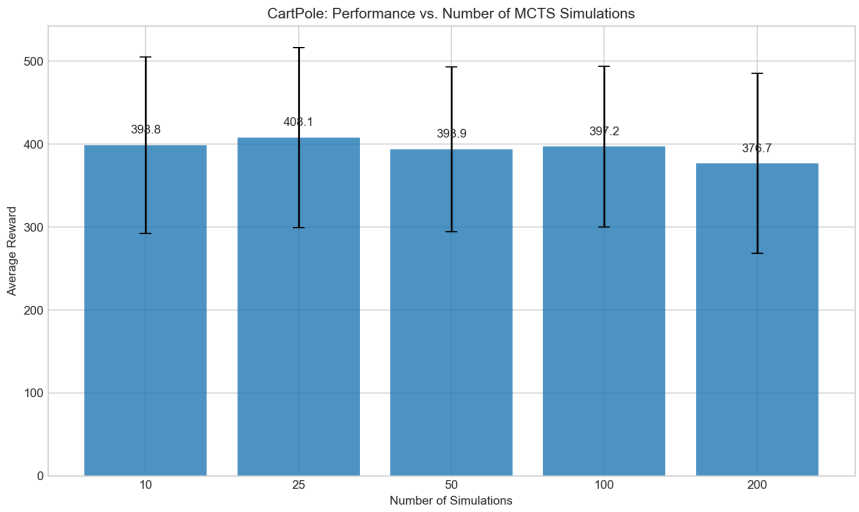

Figure 22: Impact of simulation count on CartPole performance. Higher simulations reduce the variance and prevent premature termination of the episode.

Figure 22 illustrates that even with a low budget of 10 simulations, the agent performs reasonably well, but increasing the budget to 50 or more ensures the pole is balanced for the full duration of the episode consistently.

3.4.2 Blackjack

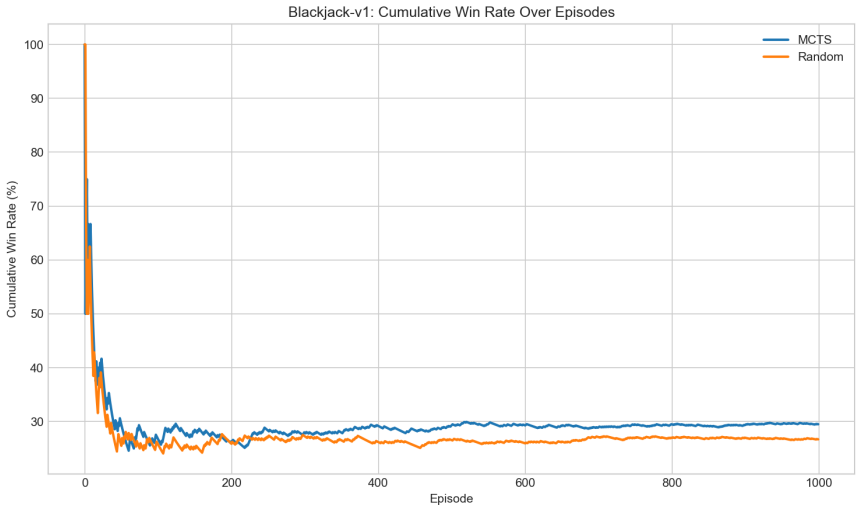

Figure 23

In the stochastic Blackjack-v1 environment, MCTS didn't achieve high score. As shown in Figure 23, the MCTS agent achieves a win rate of approximately 30%, which is has not much improvements. Maybe because Blackjack is a Stochastic environment.

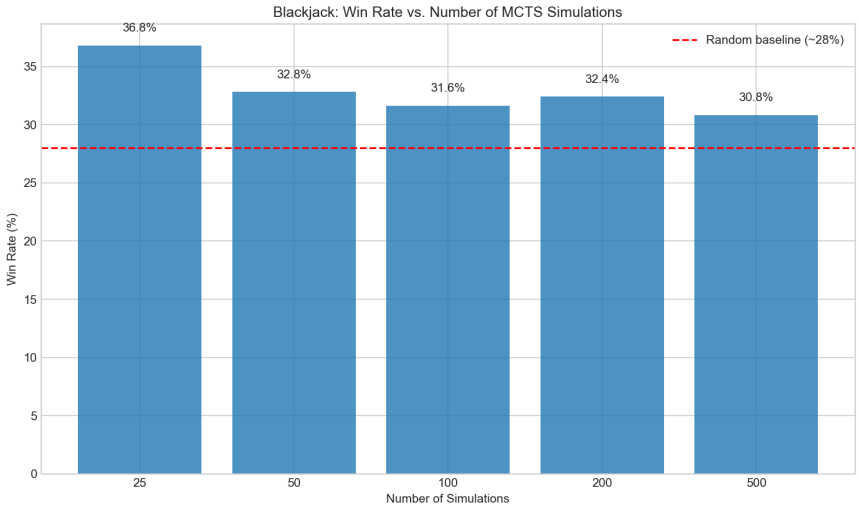

Figure 24: Win rate in Blackjack as a function of MCTS simulations. Diminishing returns are observed after 100 simulations.

The hyperparameter study in Figure 24 confirms that while planning helps, the stochastic nature of the dealer's hand limits the maximum achievable win rate. Increasing simulations beyond 100 yields diminishing returns, suggesting that the agent has successfully identified the best statistical moves (Basic Strategy) given the visible information.